Final Project - Spotify Popularity Song Prediction

Team 1

2026-04-02

```
#load libraries and set theme with Spotify colors
library(janitor)
```

```
##
## Attaching package: 'janitor'
```

```
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(C50)
library(class)
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(neuralnet)
```

```
##
## Attaching package: 'neuralnet'

## The following object is masked from 'package:dplyr':
##
##     compute
```

```
library(kernlab)
```

```
##
## Attaching package: 'kernlab'

## The following object is masked from 'package:ggplot2':
##
##     alpha
```

```
library(ggplot2)

theme_spotify <- function() {
  theme_minimal(base_size = 12) +
    theme(
      plot.background = element_rect(fill = "#121212", color = NA),
      panel.background = element_rect(fill = "#181818", color = NA),
      panel.border = element_blank(),
      panel.grid.major = element_line(color = "#2A2A2A", linewidth = 0.4),
      panel.grid.minor = element_blank(),
      axis.text = element_text(color = "#B3B3B3"),
      axis.title = element_text(color = "#F5F5F5", face = "bold"),
      plot.title = element_text(color = "#1DB954", face = "bold", size = 16),
      plot.subtitle = element_text(color = "#D9D9D9", size = 11),
      plot.caption = element_text(color = "#B3B3B3", size = 9),
      legend.background = element_rect(fill = "#181818", color = NA),
      legend.box.background = element_rect(fill = "#181818", color = NA),
      legend.key = element_rect(fill = "#181818", color = NA),
      legend.text = element_text(color = "#F5F5F5"),
      legend.title = element_text(color = "#1DB954", face = "bold"),
      strip.background = element_rect(fill = "#1DB954", color = NA),
      strip.text = element_text(color = "#121212", face = "bold")
    )
}
```

Introduction

The purpose of this project is to predict whether a Spotify song will be classified as popular or not popular using measurable song characteristics. In a business setting, this type of model can help music platforms, production companies, and music marketers understand which song attributes are associated with stronger audience appeal. The analysis uses data obtained from Kaggle and machine learning techniques to identify patterns linked to popularity outcomes.

The predicting factors in this project are the song-level variables that may help explain popularity, such as acousticness, danceability, energy, loudness, speechiness, valence, tempo, and genre-related indicators. These variables capture different dimensions of how a song sounds, feels, and is categorized, making them reasonable candidates for a popularity prediction task. The intent is to test whether combinations of these musical features can separate more popular songs from less popular ones.

We begin by building several first-level classification models, including logistic regression, k-nearest neighbors (kNN), decision trees, random forests, artificial neural networks (ANN), and support vector machines (SVM). Beyond the first-level models, we implement a stacked modeling approach. Specifically, we use the predicted probabilities from the first-level models as inputs into a second-level model. This stacked model is estimated using a decision tree framework, allowing it to learn how to optimally combine the strengths of the individual models.

From a managerial perspective, we place greater emphasis on correctly identifying popular songs than on avoiding false positives. Missing a truly popular song may lead to lost opportunities for recommendation exposure, playlist placement, and user engagement, whereas overestimating a song's popularity results in relatively smaller inefficiencies.

Getting Data

The dataset used in this analysis contains a wide range of Spotify song-level characteristics for 232,725 songs. Each observation represents an individual track and includes information on genre, as well as several acoustic features such as danceability, energy, loudness, tempo, valence, and instrumentalness. I

The primary outcome variable is a song's popularity score, which reflects its relative performance on the platform. To frame this as a classification problem, we convert the continuous popularity measure into a binary variable, where songs with popularity at or above the median are labeled as "popular," and those below the median are labeled as "not popular."

```
spotify <- read.csv("SpotifyFeatures.csv", stringsAsFactors = TRUE)
```

Processing and Cleaning Data

Prior to modeling, we perform several data preparation steps. We check for missing values and find none in the data set. We remove unique identifiers such as track ID and track name, as these variables do not provide predictive value. We also remove artist name from the dataset. We acknowledge that artist can be a valuable variable in determining a song's popularity, but due to the complexity of adding several unique dummy variables in the analysis to cover each unique artist, we determined it was best to remove it. Doing so also ensures that we rely on the characteristics of the song, rather than the artists themselves.

We also convert categorical variables, including genre and musical attributes, into dummy variables to enable their use in machine learning models. Finally, for models sensitive to feature scale, such as kNN, neural networks, and SVM, we apply min-max normalization. To prevent information leakage, the minimum and maximum values used for scaling are calculated exclusively from the training data and then applied to both the training and test sets."

We also change our variable of interest, popularity. A key transformation in this dataset is converting popular into a binary variable. Converting the outcome to a binary classification ultimately allows us to compare

our models across each other via confusion matrices that describe their accuracy, kappa, specificity, and sensitivity. For this study, we calculate the median popularity and use this as the threshold. Anything at or above the median is considered popular (1) in our original dataset, everything else is considered not popular (0).

```
summary(spotify)
```

```
##         genre                          artist_name       track_name
##  Comedy    :  9681   Giuseppe Verdi         :  1394   Home   :   100
##  Soundtrack:  9646   Giacomo Puccini        :  1137   You    :    71
##  Indie     :  9543   Kimbo Children's Music :   971   Intro  :    69
##  Jazz      :  9441   Nobuo Uematsu          :   825   Stay   :    63
##  Pop       :  9386   Richard Wagner         :   804   Wake Up:    59
##  Electronic:  9377   Wolfgang Amadeus Mozart:   800   Closer :    58
##  (Other)   :175651   (Other)                :226794   (Other):232305
##                   track_id        popularity      acousticness
##  0UE0RhnRaEYsiYgXpyLoZc:     8   Min.   :  0.00   Min.   :0.0000
##  0wY9rA9fJkuESyYm9uzVK5:     8   1st Qu.: 29.00   1st Qu.:0.0376
##  3R73Y7X53MIQZWnKloWq5i:     8   Median : 43.00   Median :0.2320
##  3uSSjnDMmoyERaAK9KvpJR:     8   Mean   : 41.13   Mean   :0.3686
##  6AIte2Iej1QKlaofpjCzW1:     8   3rd Qu.: 55.00   3rd Qu.:0.7220
##  6sVQNUvcVFTXvlk3ec0ngd:     8   Max.   :100.00   Max.   :0.9960
##  (Other)               :232677
##   danceability     duration_ms          energy         instrumentalness
##  Min.   :0.0569   Min.   :  15387   Min.   :2.03e-05   Min.   :0.0000000
##  1st Qu.:0.4350   1st Qu.: 182857   1st Qu.:3.85e-01   1st Qu.:0.0000000
##  Median :0.5710   Median : 220427   Median :6.05e-01   Median :0.0000443
##  Mean   :0.5544   Mean   : 235122   Mean   :5.71e-01   Mean   :0.1483012
##  3rd Qu.:0.6920   3rd Qu.: 265768   3rd Qu.:7.87e-01   3rd Qu.:0.0358000
##  Max.   :0.9890   Max.   :5552917   Max.   :9.99e-01   Max.   :0.9990000
##
##       key           liveness          loudness          mode
##  C      :27583   Min.   :0.00967   Min.   :-52.457   Major:151744
##  G      :26390   1st Qu.:0.09740   1st Qu.:-11.771   Minor: 80981
##  D      :24077   Median :0.12800   Median : -7.762
##  C#     :23201   Mean   :0.21501   Mean   : -9.570
##  A      :22671   3rd Qu.:0.26400   3rd Qu.: -5.501
##  F      :20279   Max.   :1.00000   Max.   :  3.744
##  (Other):88524
##   speechiness         tempo        time_signature    valence
##  Min.   :0.0222   Min.   : 30.38   0/4:     8     Min.   :0.0000
##  1st Qu.:0.0367   1st Qu.: 92.96   1/4:  2608     1st Qu.:0.2370
##  Median :0.0501   Median :115.78   3/4: 24111     Median :0.4440
##  Mean   :0.1208   Mean   :117.67   4/4:200760     Mean   :0.4549
##  3rd Qu.:0.1050   3rd Qu.:139.05   5/4:  5238     3rd Qu.:0.6600
##  Max.   :0.9670   Max.   :242.90                  Max.   :1.0000
##
```

```
str(spotify)
```

```
## 'data.frame':    232725 obs. of  18 variables:
##  $ genre           : Factor w/ 27 levels "A Capella","Alternative",..: 16 16 16 16 16 16 16 16 16 16
##  $ artist_name     : Factor w/ 14564 levels "!!!","¡MAYDAY!",..: 5255 8326 6544 5255 4112 5255 8326 7
```

```
##  $ track_name      : Factor w/ 148615 levels "____45_____",..: 18615 93903 32611 31339 91776 69420 9
##  $ track_id        : Factor w/ 176774 levels "00021Wy6AyMbLP2tqij86e",..: 4971 4768 5608 8233 10160
##  $ popularity      : int  0 1 3 0 4 0 2 15 0 10 ...
##  $ acousticness    : num  0.611 0.246 0.952 0.703 0.95 0.749 0.344 0.939 0.00104 0.319 ...
##  $ danceability    : num  0.389 0.59 0.663 0.24 0.331 0.578 0.703 0.416 0.734 0.598 ...
##  $ duration_ms     : int  99373 137373 170267 152427 82625 160627 212293 240067 226200 152694 ...
##  $ energy          : num  0.91 0.737 0.131 0.326 0.225 0.0948 0.27 0.269 0.481 0.705 ...
##  $ instrumentalness: num  0 0 0 0 0.123 0 0 0 0.00086 0.00125 ...
##  $ key             : Factor w/ 12 levels "A","A#","B","C",..: 5 10 4 5 9 5 5 10 4 11 ...
##  $ liveness        : num  0.346 0.151 0.103 0.0985 0.202 0.107 0.105 0.113 0.0765 0.349 ...
##  $ loudness        : num  -1.83 -5.56 -13.88 -12.18 -21.15 ...
##  $ mode            : Factor w/ 2 levels "Major","Minor": 1 2 2 1 1 1 1 1 1 1 ...
##  $ speechiness     : num  0.0525 0.0868 0.0362 0.0395 0.0456 0.143 0.953 0.0286 0.046 0.0281 ...
##  $ tempo           : num  167 174 99.5 171.8 140.6 ...
##  $ time_signature  : Factor w/ 5 levels "0/4","1/4","3/4",..: 4 4 5 4 4 4 4 4 4 4 ...
##  $ valence         : num  0.814 0.816 0.368 0.227 0.39 0.358 0.533 0.274 0.765 0.718 ...
```

```r
#checking for NAs
colSums(is.na(spotify))
```

```
##            genre      artist_name       track_name         track_id
##                0                0                0                0
##       popularity     acousticness     danceability      duration_ms
##                0                0                0                0
##           energy instrumentalness              key         liveness
##                0                0                0                0
##         loudness             mode      speechiness            tempo
##                0                0                0                0
##   time_signature          valence
##                0                0
```

```r
#remove unique ID
spotify$track_id <- NULL
spotify$track_name <- NULL
#saving as separate vector in case want to integrate
artistname <- spotify$artist_name
spotify$artist_name <- NULL

#make factors into dummies
spotify_dummy <- as.data.frame(model.matrix(~ . -1, data=spotify))

#clean names
spotify_dummy <- clean_names(spotify_dummy)
```

```r
median_popularity <- median(spotify$popularity, na.rm = TRUE)

#create a new binary variable called popular_binary - a song gets value 1 if its popularity is at or ab
spotify_dummy <- spotify_dummy %>%
  mutate(popular_binary = if_else(popularity >= median_popularity, 1, 0)) %>%
  select(-popularity)
```

Examining Data

Before we continue, it would be helpful to further examine our data to understand its underlying trends. The purpose of this is to better understand the characteristics of the data before moving into predictive modeling. The code checks whether the target classes are balanced, reviews the composition of the genre variable, and visualizes how song features vary across categories.

First, we examine class balance. The table shows that there are 114,070 "not popular" (0) songs and 118,655 "popular" (1) songs, which corresponds to proportions of 49.0% and 51.0%, respectively.

```
table(spotify_dummy$popular_binary)
```

```
##
##      0      1
## 114070 118655
```

```
prop.table(table(spotify_dummy$popular_binary))
```

```
##
##         0         1
## 0.4901493 0.5098507
```

Second, we examine the summary statistics. The output shows that the median danceability is 0.571, median energy is 0.605, median loudness is about -7.76, and median duration is about 220,427 milliseconds, or roughly 3.7 minutes. These values suggest the dataset is centered on songs with moderate-to-high rhythmic accessibility and energy, while still covering a broad range from very acoustic to highly energetic tracks.

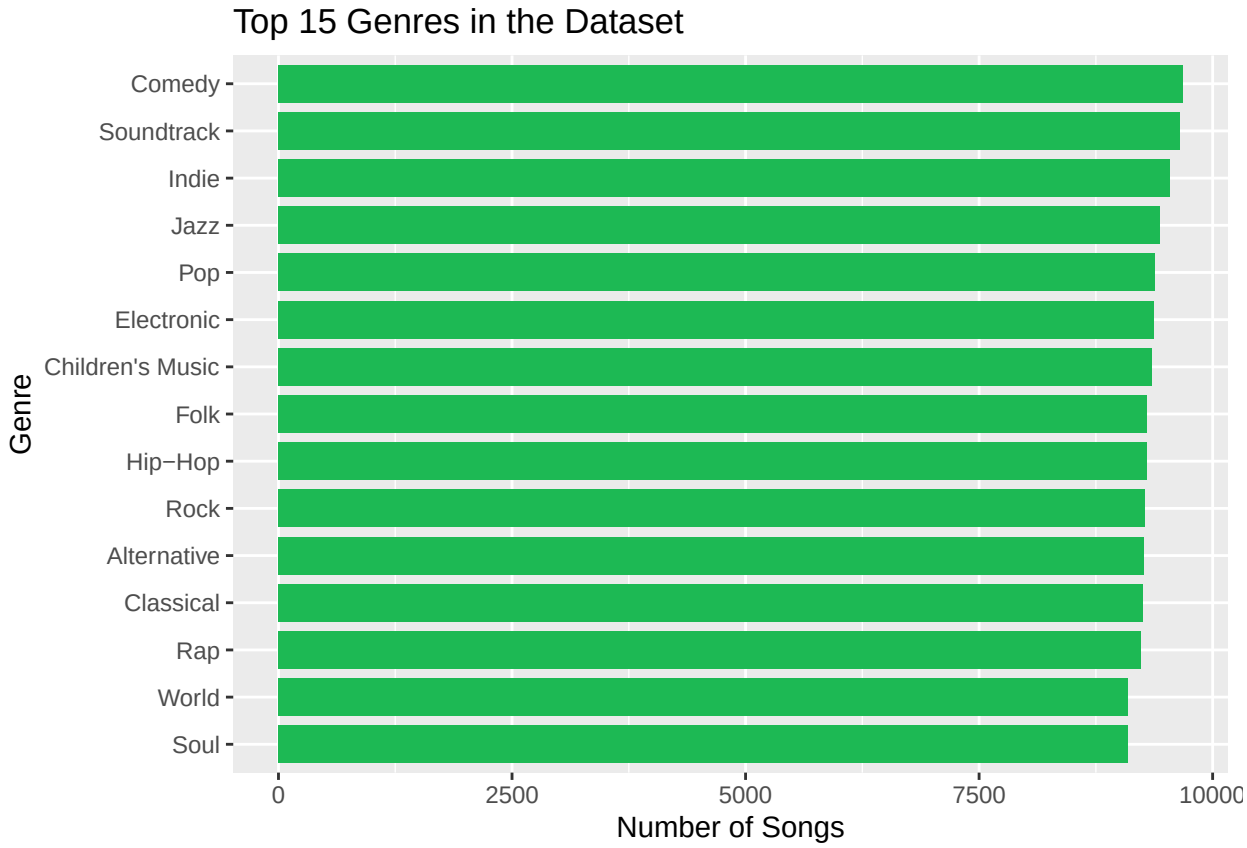
```
summary(
  spotify %>%
    dplyr::select(
      acousticness, danceability, duration_ms, energy, instrumentalness,
      liveness, loudness, speechiness, tempo, valence, popularity)
)
```

```
##   acousticness     danceability     duration_ms          energy
##  Min.   :0.0000   Min.   :0.0569   Min.   :  15387   Min.   :2.03e-05
##  1st Qu.:0.0376   1st Qu.:0.4350   1st Qu.: 182857   1st Qu.:3.85e-01
##  Median :0.2320   Median :0.5710   Median : 220427   Median :6.05e-01
##  Mean   :0.3686   Mean   :0.5544   Mean   : 235122   Mean   :5.71e-01
##  3rd Qu.:0.7220   3rd Qu.:0.6920   3rd Qu.: 265768   3rd Qu.:7.87e-01
##  Max.   :0.9960   Max.   :0.9890   Max.   :5552917   Max.   :9.99e-01
##  instrumentalness       liveness          loudness        speechiness
##  Min.   :0.0000000   Min.   :0.00967   Min.   :-52.457   Min.   :0.0222
##  1st Qu.:0.0000000   1st Qu.:0.09740   1st Qu.:-11.771   1st Qu.:0.0367
##  Median :0.0000443   Median :0.12800   Median : -7.762   Median :0.0501
##  Mean   :0.1483012   Mean   :0.21501   Mean   : -9.570   Mean   :0.1208
##  3rd Qu.:0.0358000   3rd Qu.:0.26400   3rd Qu.: -5.501   3rd Qu.:0.1050
##  Max.   :0.9990000   Max.   :1.00000   Max.   :  3.744   Max.   :0.9670
##      tempo            valence         popularity
##  Min.   : 30.38   Min.   :0.0000   Min.   :  0.00
##  1st Qu.: 92.96   1st Qu.:0.2370   1st Qu.: 29.00
##  Median :115.78   Median :0.4440   Median : 43.00
```

```
##  Mean   :117.67   Mean   :0.4549   Mean   : 41.13
##  3rd Qu.:139.05   3rd Qu.:0.6600   3rd Qu.: 55.00
##  Max.   :242.90   Max.   :1.0000   Max.   :100.00
```

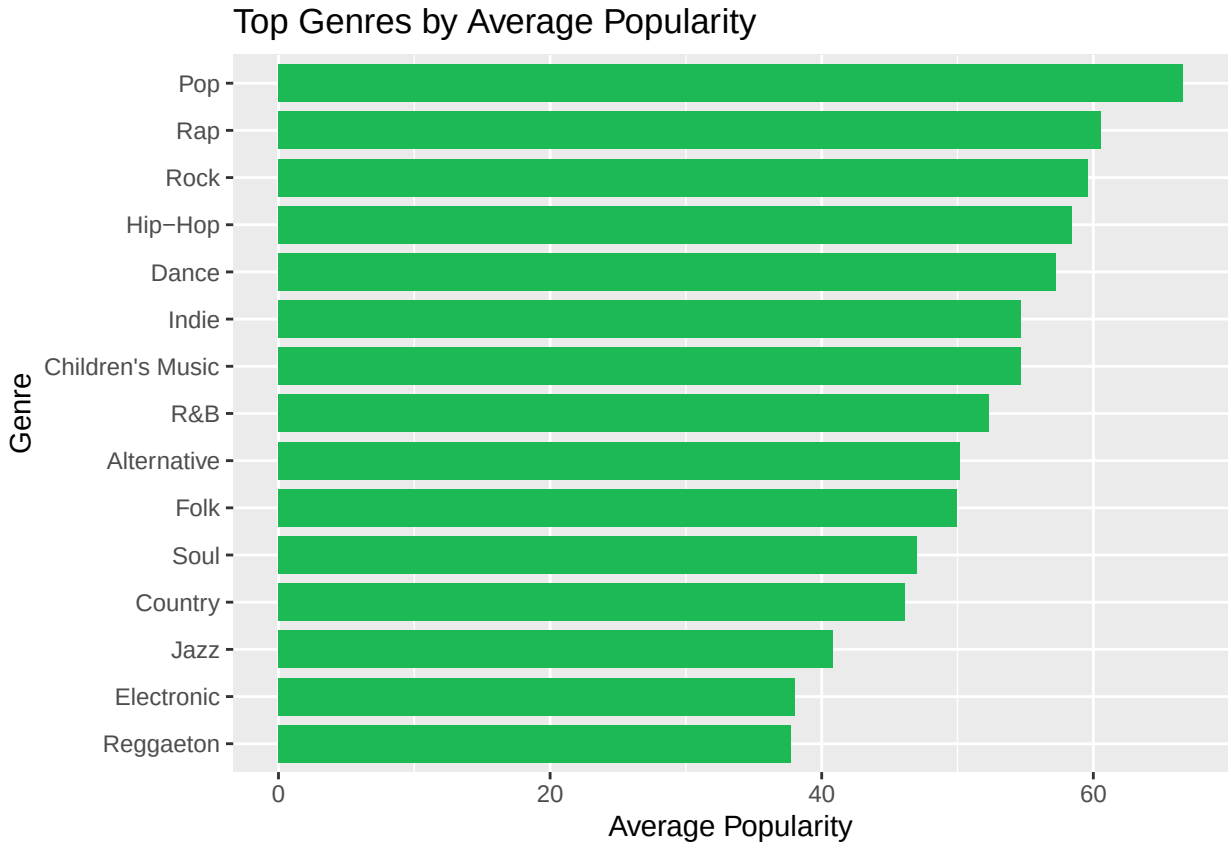
Next we examine the top 15 genres and plot them in a bar plot. The purpose of this is to determine which categories dominate the dataset. This matters from an analysis standpoint because if there is a highly concentrated number of songs from one or two genres, those categories can disproportionately shape the results. The graph shows that the top 15 genres are similar in number of songs.

```r
#finding the most frequent genres
top_genres <- spotify %>%
  dplyr::count(genre, sort = TRUE) %>%
  dplyr::slice_head(n = 15)

#plotting the top genres
ggplot(top_genres, aes(x = reorder(genre, n), y = n)) +
  geom_col(fill = "#1DB954", width = 0.8) +
  coord_flip() +
  labs(
    title = "Top 15 Genres in the Dataset",
    x = "Genre", y = "Number of Songs"
  )
```



We also look at the top 15 genres by popularity to understand which genres tend to be the most popular, regardless of frequency/count. The graph shows that, on average, Pop, Rap and Rock songs have the highest popularity.

```r
#comparing average popularity across genres
genre_popularity <- spotify %>%
  dplyr::group_by(genre) %>%
  dplyr::summarise(
    avg_popularity = mean(popularity, na.rm = TRUE),
    count = dplyr::n(), .groups = "drop"
  ) %>%
  dplyr::arrange(desc(avg_popularity)) %>%
  dplyr::slice_head(n = 15)

ggplot(genre_popularity, aes(x = reorder(genre, avg_popularity), y = avg_popularity)) +
  geom_col(fill = "#1DB954", width = 0.8) +
  coord_flip() +
  labs(
    title = "Top Genres by Average Popularity",
    x = "Genre", y = "Average Popularity"
  )
```



Next we examine the data on genre-level. The table below aggregates average values of danceability, energy, valence, acousticness, loudness, tempo and count to see what trends there are among the different genres. This helps us see that each genre carries distinct characteristics: for instance, some genres are more energetic and danceable, while others are more acoustic. Note that in the output, only 6 genres are shown.

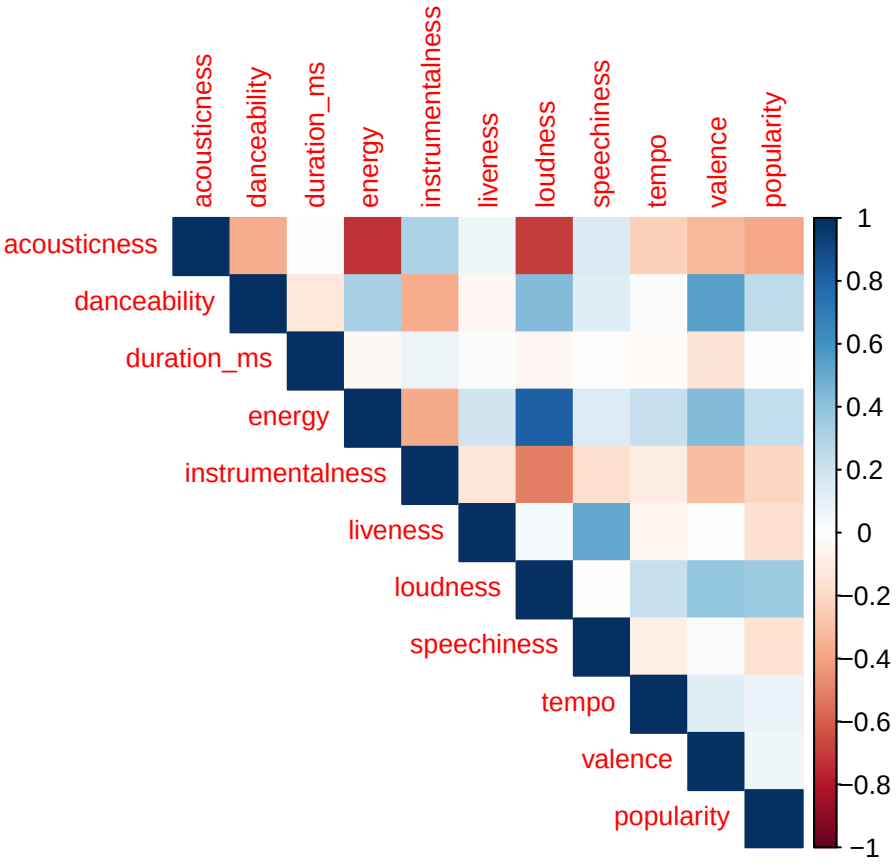
```r
genre_feature_summary <- spotify %>%
  dplyr::group_by(genre) %>%
  dplyr::summarise(
    danceability = mean(danceability, na.rm = TRUE),
```

```
    energy = mean(energy, na.rm = TRUE),
    valence = mean(valence, na.rm = TRUE),
    acousticness = mean(acousticness, na.rm = TRUE),
    loudness = mean(loudness, na.rm = TRUE),
    tempo = mean(tempo, na.rm = TRUE),
    count = dplyr::n(), .groups = "drop"
  )

head(genre_feature_summary)
```

```
## # A tibble: 6 x 8
##   genre            danceability energy valence acousticness loudness tempo count
##   <fct>                   <dbl>  <dbl>   <dbl>        <dbl>    <dbl> <dbl> <int>
## 1 A Capella               0.412  0.250   0.329        0.830   -13.7   112.   119
## 2 Alternative             0.542  0.712   0.450        0.162    -6.54  123.  9263
## 3 Anime                   0.472  0.665   0.442        0.287    -7.92  127.  8936
## 4 Blues                   0.528  0.606   0.579        0.328    -9.05  121.  9023
## 5 Children's Music        0.697  0.397   0.676        0.592   -11.6   121.  5403
## 6 Children's Music        0.542  0.707   0.449        0.163    -6.53  122.  9353
```

Finally, we create a correlation matrix to understand which song characteristics are most correlated with each other.

```
numeric_features <- spotify %>%
  dplyr::select(
    acousticness, danceability, duration_ms, energy, instrumentalness,
    liveness, loudness, speechiness, tempo, valence, popularity)

cor_matrix <- cor(numeric_features, use = "complete.obs")

corrplot::corrplot(cor_matrix, method = "color", type = "upper", tl.cex = 0.8)
```



Key Takeaways from Exploratory Analysis The exploratory data analysis suggests that the prediction problem is well-structured because the target classes are very balanced. After converting popularity into a binary outcome, the dataset contains about 49.0% "not popular" songs and 51.0% "popular" songs, so neither class dominates the sample. This is important from a business perspective because it means later model results are less likely to be distorted by class imbalance. In other words, if a model performs well, that performance is more likely to reflect real differences in song characteristics rather than simply benefiting from an uneven split in the outcome variable. The summary statistics also show that the typical song in the dataset is moderately danceable, moderately energetic, and about 3.7 minutes long, which gives a useful baseline for understanding what a "normal" track looks like in this sample.

A second major takeaway is that the dataset is not heavily concentrated in just one genre, at least among the top categories. The report notes that the top 15 genres appear relatively similar in number of songs, which is helpful because it reduces the risk that one genre overwhelmingly drives the analysis. At the same time, the genre-level summaries show that musical features vary meaningfully across categories. For example, classical songs have much lower danceability and energy but much higher acousticness, while children's music appears more upbeat and accessible on average. This tells us that genre is likely to matter in predicting popularity, not only because genres differ in audience appeal, but also because they capture underlying differences in the musical profile of songs.

Finally, the exploratory data analysis shows that popularity is probably influenced by a combination of musical attributes rather than any single factor alone. The genre comparisons indicate that some genres tend to be more energetic and danceable, while others are more acoustic or subdued, so the model will likely need to recognize patterns across several features at once. For a manager, the practical implication is that popularity does not appear random: it is connected to measurable song characteristics and to the type of music being produced. That makes the dataset suitable for predictive modeling and supports the broader business value of the project, since the analysis suggests it should be possible to identify the kinds of songs that are more likely to become popular.

Splitting Data

To reduce computational demands while maintaining a sufficiently large sample for model estimation, we randomly select 20% of the full dataset for predictive modeling. This produces a modeling sample of approximately 46,545 songs. We then randomly divide this modeling sample approximately evenly between training and test sets, resulting in 23,272 training observations and 23,273 test observations. We maintain both non-scaled and scaled versions of the modeling data. Non-scaled data are used for logistic regression, decision trees, and random forests, while scaled data are used for kNN, neural networks, and SVM. The same sampled observations and train/test indices are used across models to ensure consistent comparisons.

```r
set.seed(12345)

# Randomly select 20% of the full dataset for modeling
model_rows <- sample(
  1:nrow(spotify_dummy),
  size = 0.20 * nrow(spotify_dummy)
)

spotify_model <- spotify_dummy[model_rows, ]

# Split modeling sample 50/50
ratio <- 0.5

train_rows <- sample(
  1:nrow(spotify_model),
  size = ratio * nrow(spotify_model)
)

spotify_train <- spotify_model[train_rows, ]
spotify_test  <- spotify_model[-train_rows, ]


# Separate outcome variable
x_train <- spotify_train[, names(spotify_train) != "popular_binary"]
x_test  <- spotify_test[, names(spotify_test) != "popular_binary"]

# Calculate min and max values using training data only
train_min <- sapply(x_train, min)
train_max <- sapply(x_train, max)

# Min-max scaling function using training-set parameters
scale_with_train <- function(df, mins, maxs) {
  as.data.frame(
    Map(
      function(x, mn, mx) {
        if (mx == mn) {
          return(rep(0, length(x)))
        }
        (x - mn) / (mx - mn)
      },
      df,
      mins,
      maxs
    )
```

```
  )
}

spotify_scale_train <- scale_with_train(
  x_train,
  train_min,
  train_max
)

spotify_scale_test <- scale_with_train(
  x_test,
  train_min,
  train_max
)

# Add outcome variable back
spotify_scale_train$popular_binary <- spotify_train$popular_binary
spotify_scale_test$popular_binary <- spotify_test$popular_binary
```

Build Models

Logistic Regression Logistic regression serves as a baseline model for this analysis. It estimates the probability that a song is popular as a function of its features, assuming a linear relationship between the predictors and the log-odds of popularity.

The logistic regression results indicate that genre is an important predictor of whether a song is classified as popular. Several genre indicators, including Alternative, Dance, Folk, Hip-Hop, Indie, Pop, Rap, and Rock, have statistically significant positive coefficients, while other genres have negative associations with popularity. Among the continuous song characteristics, danceability and loudness have statistically significant positive relationships with popularity, while instrumentalness, liveness, and valence have statistically significant negative relationships. However, several other characteristics, including acousticness, duration, energy, and tempo, are not statistically significant after controlling for the other predictors. These results suggest that popularity is associated with a combination of genre and selected musical characteristics rather than every available audio feature

```
logistic_model <- glm(popular_binary ~ ., data=spotify_train, family = "binomial")
summary(logistic_model)
```

```
##
## Call:
## glm(formula = popular_binary ~ ., family = "binomial", data = spotify_train)
##
## Coefficients: (1 not defined because of singularities)
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)              7.616e+00  5.354e+02   0.014 0.988650
## genre_a_capella         -1.225e+01  1.483e+02  -0.083 0.934156
## genre_alternative        3.196e+00  1.346e-01  23.749  < 2e-16 ***
## genre_anime             -1.491e+00  1.709e-01  -8.725  < 2e-16 ***
## genre_blues             -2.448e-01  1.245e-01  -1.966 0.049249 *
## genre_childrens_music   -3.811e+00  5.879e-01  -6.482 9.06e-11 ***
## genre_children_s_music   5.360e+00  2.833e-01  18.917  < 2e-16 ***
## genre_classical         -4.181e-01  1.493e-01  -2.800 0.005116 **
## genre_comedy            -1.800e+00  3.141e-01  -5.732 9.94e-09 ***
```

```
## genre_country           1.555e+00  1.122e-01  13.860  < 2e-16 ***
## genre_dance             4.883e+00  2.438e-01  20.031  < 2e-16 ***
## genre_electronic        3.409e-01  1.162e-01   2.935 0.003333 **
## genre_folk              2.822e+00  1.215e-01  23.233  < 2e-16 ***
## genre_hip_hop           6.598e+00  5.114e-01  12.903  < 2e-16 ***
## genre_indie             5.892e+00  3.458e-01  17.038  < 2e-16 ***
## genre_jazz              9.189e-01  1.106e-01   8.305  < 2e-16 ***
## genre_movie            -2.013e+00  2.190e-01  -9.193  < 2e-16 ***
## genre_opera            -4.036e+00  5.872e-01  -6.874 6.25e-12 ***
## genre_pop               6.429e+00  4.575e-01  14.054  < 2e-16 ***
## genre_r_b               3.092e+00  1.343e-01  23.029  < 2e-16 ***
## genre_rap               7.983e+00  1.005e+00   7.940 2.03e-15 ***
## genre_reggae            1.135e-01  1.221e-01   0.930 0.352389
## genre_reggaeton         3.525e-01  1.217e-01   2.896 0.003783 **
## genre_rock              6.214e+00  4.188e-01  14.836  < 2e-16 ***
## genre_ska              -8.170e-01  1.438e-01  -5.683 1.33e-08 ***
## genre_soul              1.734e+00  1.117e-01  15.514  < 2e-16 ***
## genre_soundtrack       -8.792e-02  1.431e-01  -0.615 0.538862
## genre_world                    NA         NA      NA       NA
## acousticness           -7.521e-02  1.015e-01  -0.741 0.458689
## danceability            5.527e-01  1.796e-01   3.077 0.002089 **
## duration_ms             2.933e-07  1.807e-07   1.623 0.104573
## energy                 -1.689e-01  1.888e-01  -0.895 0.370815
## instrumentalness       -3.900e-01  9.098e-02  -4.286 1.82e-05 ***
## key_a_number           -8.533e-02  1.049e-01  -0.813 0.416102
## key_b                  -1.730e-01  1.003e-01  -1.724 0.084666 .
## key_c                  -9.254e-02  9.100e-02  -1.017 0.309182
## key_c_number           -1.515e-02  9.753e-02  -0.155 0.876589
## key_d                  -4.678e-02  9.469e-02  -0.494 0.621236
## key_d_number           -1.925e-02  1.361e-01  -0.141 0.887475
## key_e                  -9.121e-02  1.002e-01  -0.910 0.362694
## key_f                  -1.119e-01  9.769e-02  -1.146 0.252002
## key_f_number           -8.728e-02  1.061e-01  -0.822 0.410853
## key_g                   1.535e-02  8.973e-02   0.171 0.864138
## key_g_number           -2.002e-02  1.072e-01  -0.187 0.851783
## liveness               -5.257e-01  1.368e-01  -3.843 0.000121 ***
## loudness                2.261e-02  8.007e-03   2.823 0.004754 **
## mode_minor              8.495e-02  4.668e-02   1.820 0.068787 .
## speechiness            -5.073e-01  2.641e-01  -1.921 0.054789 .
## tempo                  -4.083e-04  7.283e-04  -0.561 0.575085
## time_signature1_4      -8.940e+00  5.354e+02  -0.017 0.986678
## time_signature3_4      -8.623e+00  5.354e+02  -0.016 0.987151
## time_signature4_4      -8.512e+00  5.354e+02  -0.016 0.987316
## time_signature5_4      -8.522e+00  5.354e+02  -0.016 0.987301
## valence                -3.215e-01  1.130e-01  -2.846 0.004432 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 32260  on 23271  degrees of freedom
## Residual deviance: 14423  on 23219  degrees of freedom
## AIC: 14529
##
```

```
## Number of Fisher Scoring iterations: 12
```

KNN The k-nearest neighbors model classifies songs based on the popularity of the most similar observations in the training data. Using a distance-based approach, kNN assigns a class based on the majority label among the k closest neighbors.

The kNN model produces predictions for both popularity classes, with 12,105 observations classified as not popular and 11,168 classified as popular. Because kNN is based on the similarity between observations rather than estimated coefficients, the model does not provide individual parameter estimates like logistic regression. Its usefulness is therefore evaluated primarily through its performance on the test data.

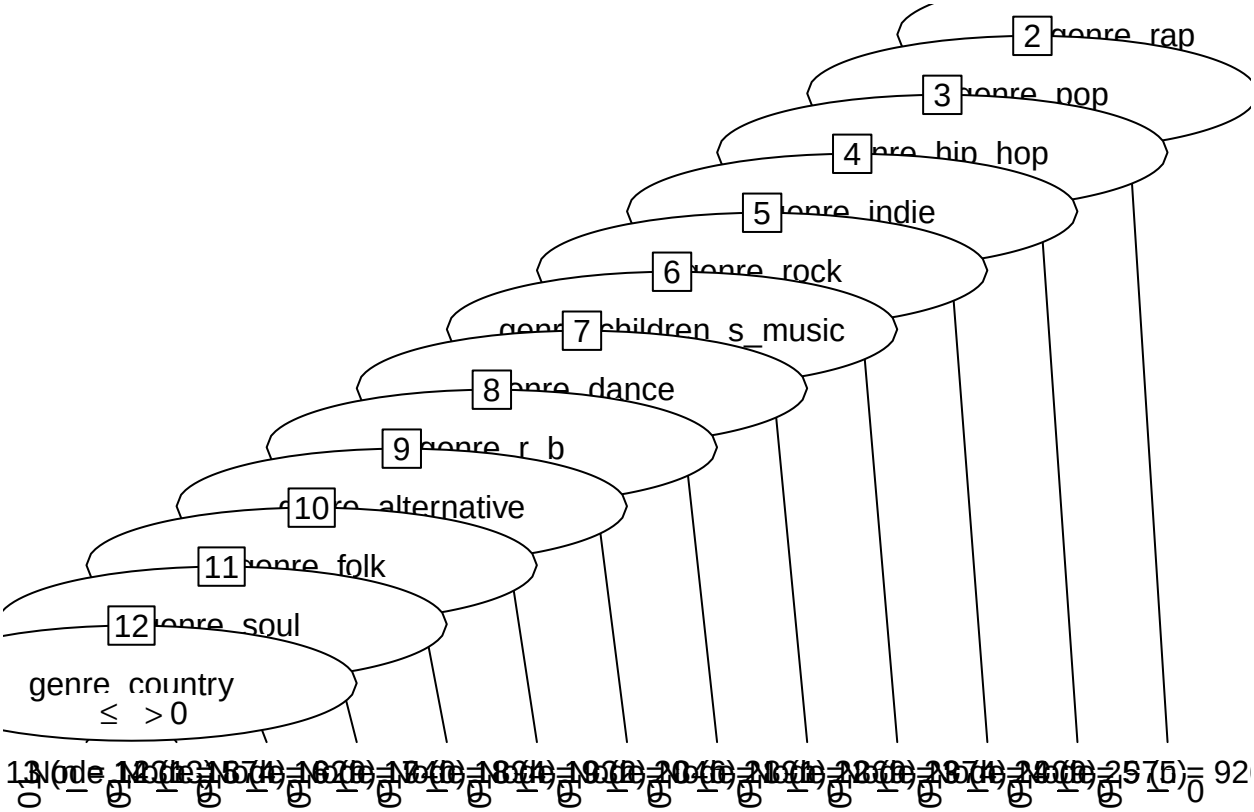
```
knn_model <- knn(
  train = spotify_scale_train[, !(names(spotify_scale_train) %in% c("popular_binary"))],
  test  = spotify_scale_test[, !(names(spotify_scale_test) %in% c("popular_binary"))],
  cl    = as.factor(spotify_scale_train$popular_binary), k = 7, prob  = TRUE)

summary(knn_model)
```

```
##     0     1
## 12105 11168
```

Decision Tree The decision tree model creates a set of decision rules that classify songs as popular or not. This approach allows us to understand which features, such as genre or acoustic attributes, are most influential in determining popularity.

The decision tree relies heavily on genre when separating popular and not-popular songs. The resulting tree contains 13 terminal decision rules, with genres such as Rap, Pop, Hip-Hop, Indie, Rock, Dance, Alternative, Folk, Soul, and Country appearing throughout the tree. The training error is approximately 13.3%, meaning the tree correctly classifies most observations in the training sample. The strong reliance on genre suggests that genre membership contains substantial predictive information about a song's popularity within this dataset

```
dt_model <- C5.0(as.factor(popular_binary) ~ ., data = spotify_train)
plot(dt_model)
```




```
summary(dt_model)
```

```
##
## Call:
## C5.0.formula(formula = as.factor(popular_binary) ~ ., data = spotify_train)
##
##
## C5.0 [Release 2.07 GPL Edition]      Sun Aug 23 19:34:35 2026
## -------------------------------
##
## Class specified by attribute 'outcome'
##
## Read 23272 cases (54 attributes) from undefined.data
##
## Decision tree:
##
## genre_rap > 0: 1 (926/1)
## genre_rap <= 0:
## :...genre_pop > 0: 1 (975/5)
##     genre_pop <= 0:
##     :...genre_hip_hop > 0: 1 (909/4)
##         genre_hip_hop <= 0:
##         :...genre_indie > 0: 1 (974/9)
##             genre_indie <= 0:
##             :...genre_rock > 0: 1 (869/6)
##                 genre_rock <= 0:
##                 :...genre_children_s_music > 0: 1 (891/14)
```

```
##                     genre_children_s_music <= 0:
##                     :...genre_dance > 0: 1 (846/20)
##                         genre_dance <= 0:
##                         :...genre_r_b > 0: 1 (932/118)
##                             genre_r_b <= 0:
##                             :...genre_alternative > 0: 1 (894/110)
##                                 genre_alternative <= 0:
##                                 :...genre_folk > 0: 1 (940/164)
##                                     genre_folk <= 0:
##                                     :...genre_soul > 0: 1 (929/343)
##                                         genre_soul <= 0:
##                                         :...genre_country <= 0: 0 (12313/1938)
##                                             genre_country > 0: 1 (874/362)
##
##
## Evaluation on training data (23272 cases):
##
##      Decision Tree
##    ----------------
##    Size      Errors
##
##      13 3094(13.3%)   <<
##
##
##     (a)   (b)    <-classified as
##    ----  ----
##   10375  1156    (a): class 0
##    1938  9803    (b): class 1
##
##
##  Attribute usage:
##
##  100.00% genre_rap
##   96.02% genre_pop
##   91.83% genre_hip_hop
##   87.93% genre_indie
##   83.74% genre_rock
##   80.01% genre_children_s_music
##   76.18% genre_dance
##   72.54% genre_r_b
##   68.54% genre_alternative
##   64.70% genre_folk
##   60.66% genre_soul
##   56.66% genre_country
##
##
## Time: 0.3 secs
```

Random Forest Random forest builds on the decision tree framework by combining many trees trained on bootstrapped samples of the data. By averaging predictions across multiple trees, the model can reduce the instability associated with an individual decision tree and improve generalization

The random forest model improves on a single decision tree by averaging predictions across 500 trees and considering a random subset of variables at each split. The model produces an out-of-bag error rate of

approximately 13.45%, indicating relatively strong performance on observations not included in individual bootstrap samples. The variable importance plot provides an additional way to examine which predictors contribute most strongly to the forest's classification decisions. Unlike the single decision tree, the random forest can incorporate information from many song characteristics simultaneously rather than relying on one fixed sequence of splits.

```
rf_model <- randomForest(as.factor(popular_binary) ~ ., data = spotify_train)

print(rf_model)
```

```
##
## Call:
##  randomForest(formula = as.factor(popular_binary) ~ ., data = spotify_train)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 7
##
##         OOB estimate of  error rate: 13.45%
## Confusion matrix:
##       0    1 class.error
## 0 10382 1149  0.09964444
## 1  1981 9760  0.16872498
```

```
rf_summary <- data.frame(
  Metric = c(
    "Number of Trees",
    "Variables Tried per Split",
    "OOB Error Rate"
  ),
  Value = c(
    rf_model$ntree,
    rf_model$mtry,
    paste0(round(rf_model$err.rate[nrow(rf_model$err.rate), "OOB"] * 100, 2), "%")
  )
)

knitr::kable(
  rf_summary,
  caption = "Random Forest Model Summary"
)
```

Table 1: Random Forest Model Summary

Metric	Value
Number of Trees	500
Variables Tried per Split	7
OOB Error Rate	13.45%

```
rf_confusion <- as.data.frame.matrix(rf_model$confusion)

knitr::kable(
```

```
  rf_confusion,
  digits = 3,
  caption = "Random Forest Out-of-Bag Confusion Matrix"
)
```

Table 2: Random Forest Out-of-Bag Confusion Matrix

	0	1	class.error
0	10382	1149	0.100
1	1981	9760	0.169

ANN The neural network model captures complex, nonlinear relationships between song features and popularity. By passing inputs through hidden layers of interconnected nodes, the model can learn patterns that are not easily captured by linear or tree-based methods.

The artificial neural network uses a hidden layer containing five nodes to capture nonlinear relationships between song characteristics and popularity. Because neural networks do not provide easily interpretable coefficients in the same way as logistic regression, the primary focus is on predictive performance rather than individual parameter interpretation. The model is therefore evaluated on the test sample using the same accuracy, sensitivity, and other classification metrics as the remaining models.

```
set.seed(12345)
ann_model <- neuralnet(
  popular_binary ~ .,
  data = spotify_scale_train,
  hidden = c(5),
  stepmax = 1e6,
  linear.output = FALSE)

plot(ann_model)
```

SVM The support vector machine uses a kernel-based approach to classify songs by finding an optimal boundary that separates popular and non-popular tracks. The radial basis function (RBF) kernel allows the model to capture nonlinear relationships in the data.

The SVM uses a radial basis function kernel, allowing it to construct a nonlinear decision boundary between popular and not-popular songs. This is useful when the relationship between song characteristics and popularity cannot be adequately represented by a simple linear boundary. As with the neural network, the model is evaluated primarily according to its performance on previously unseen test observations rather than through individual coefficient estimates.

```
set.seed(12345)
svmrbfdot_model <- ksvm(as.factor(popular_binary) ~ ., data = spotify_scale_train, kernel = "rbfdot", pr
```

Generating Predictions

Using the trained models, we generate predictions on the test dataset to evaluate out-of-sample performance. For each model, we obtain predicted probabilities that a song is classified as "popular." To facilitate evaluation, we also convert predicted probabilities into binary classifications using a threshold of 0.5, where songs with predicted probabilities above the threshold are classified as popular. For models that directly produce class labels, such as kNN and decision trees, we extract both the predicted classes and the associated probabilities where available.

```r
logistic_predictions <- predict(logistic_model, spotify_test, type = "response")
logistic_binpred <- ifelse(logistic_predictions >= 0.5, 1, 0)
```

Logistic Regression

```r
knn_prob <- attr(knn_model, "prob")
knn_predictions <- ifelse(as.character(knn_model) == "1", knn_prob, 1 - knn_prob)
summary(knn_predictions)
```

KNN

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  0.1429  0.4286  0.5097  1.0000  1.0000
```

```r
rm(knn_prob)

knn_binpred <- knn_model
```

```r
dt_pred <- predict(dt_model, spotify_test,  type = "prob")
dt_predictions <- dt_pred[, "1"]
summary(dt_predictions)
```

Decision Tree

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.1574  0.1574  0.1574  0.5041  0.9758  0.9984
```

```r
dt_binpred <- predict(dt_model, spotify_test)
```

```r
rf_pred <- predict(rf_model,  spotify_test, type = "prob")
rf_predictions <- rf_pred[, "1"]
summary(rf_predictions)
```

Random Forest

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  0.1400  0.4000  0.5022  0.9320  1.0000
```

```r
rf_binpred <- predict(rf_model,  spotify_test)
```

```
ann_pred <- neuralnet::compute(
  ann_model,
  spotify_scale_test[, names(spotify_scale_test) != "popular_binary"])$net.result

ann_predictions <- as.vector(ann_pred)
summary(ann_predictions)
```

ANN

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.01460 0.09919 0.42790 0.50462 0.98037 0.99176
```

```
ann_binpred <- ifelse(ann_predictions > 0.5, 1, 0)
```

```
svm_pred <- predict(
  svmrbfdot_model, spotify_scale_test[, names(spotify_scale_test) != "popular_binary"],
  type = "prob")

svm_binpred <- predict(
  svmrbfdot_model, spotify_scale_test[, names(spotify_scale_test) != "popular_binary"],
  type = "response")

svm_predictions <- svm_pred[, "1"]
```

SVM The resulting outputs represent each model's estimated probability or predicted classification for observations in the test set. These predictions are generated from the same test observations across all six models, allowing their performance to be compared directly. For probability-based models, a threshold of 0.50 is used to convert predicted probabilities into binary classifications, while models such as kNN and the decision tree can directly return predicted classes.

Evaluate Models

Logistic regression achieves an accuracy of approximately 86.5%, with a sensitivity of 82.5% and specificity of 90.7%. This means the model correctly identifies approximately 82.5% of songs that are actually popular while correctly identifying about 90.7% of songs that are not popular. Its Kappa value of 0.730 also indicates substantial agreement between the predicted and actual classifications beyond what would be expected by chance

```
#logistic regression
logistic_confus <- confusionMatrix(
  factor(logistic_binpred, levels = c(0, 1)),
  factor(spotify_test$popular_binary, levels = c(0, 1)),
  positive = "1")

logistic_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction     0     1
##          0 10279  2084
##          1  1059  9851
##
##                Accuracy : 0.865
##                  95% CI : (0.8605, 0.8693)
##     No Information Rate : 0.5128
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7303
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8254
##             Specificity : 0.9066
##          Pos Pred Value : 0.9029
##          Neg Pred Value : 0.8314
##              Prevalence : 0.5128
##          Detection Rate : 0.4233
##    Detection Prevalence : 0.4688
##       Balanced Accuracy : 0.8660
##
##        'Positive' Class : 1
##
```

The kNN model achieves approximately 84.9% accuracy, with sensitivity of 82.1% and specificity of 87.9%. Its overall performance is slightly weaker than logistic regression, particularly in terms of its ability to correctly identify not-popular songs. Nevertheless, the model maintains relatively balanced performance between the two outcome classes

```
#knn
knn_confus <- confusionMatrix(
  factor(knn_binpred, levels = c(0, 1)),
  factor(spotify_test$popular_binary, levels = c(0, 1)),
  positive = "1")

knn_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 9966 2139
##          1 1372 9796
##
##                Accuracy : 0.8491
##                  95% CI : (0.8445, 0.8537)
##     No Information Rate : 0.5128
##     P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                  Kappa : 0.6986
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8208
##             Specificity : 0.8790
##          Pos Pred Value : 0.8771
##          Neg Pred Value : 0.8233
##              Prevalence : 0.5128
##          Detection Rate : 0.4209
##    Detection Prevalence : 0.4799
##       Balanced Accuracy : 0.8499
##
##        'Positive' Class : 1
##
```

The decision tree achieves approximately 86.5% accuracy, making its overall performance very similar to logistic regression. Its sensitivity is approximately 82.7%, while its specificity is approximately 90.5%. Among the first-level models, the decision tree produces one of the highest sensitivity values, which is particularly relevant because correctly identifying popular songs is a priority in this analysis.

```r
#decision tree
dt_confus <- confusionMatrix(
  factor(dt_binpred, levels = c(0, 1)),
  factor(spotify_test$popular_binary, levels = c(0, 1)),
  positive = "1"
)
dt_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction     0     1
##          0 10259  2066
##          1  1079  9869
##
##                Accuracy : 0.8649
##                  95% CI : (0.8604, 0.8692)
##     No Information Rate : 0.5128
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7301
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8269
##             Specificity : 0.9048
##          Pos Pred Value : 0.9014
##          Neg Pred Value : 0.8324
##              Prevalence : 0.5128
##          Detection Rate : 0.4241
##    Detection Prevalence : 0.4704
##       Balanced Accuracy : 0.8659
```

```
##
##        'Positive' Class : 1
##
```

Random forest achieves approximately 86.4% accuracy, with sensitivity of 82.1% and specificity of 90.9%. Although random forests are generally designed to improve on individual decision trees by combining many trees, its performance in this analysis is very similar to that of the simpler decision tree and logistic regression models. This suggests that the additional model complexity produces only limited improvement for this particular classification problem

```
#random forest
rf_confus <- confusionMatrix(
  factor(rf_binpred, levels = c(0, 1)),
  factor(spotify_test$popular_binary, levels = c(0, 1)),
  positive = "1"
)
rf_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction     0     1
##          0 10307  2131
##          1  1031  9804
##
##                Accuracy : 0.8641
##                  95% CI : (0.8597, 0.8685)
##     No Information Rate : 0.5128
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7287
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8214
##             Specificity : 0.9091
##          Pos Pred Value : 0.9048
##          Neg Pred Value : 0.8287
##              Prevalence : 0.5128
##          Detection Rate : 0.4213
##    Detection Prevalence : 0.4656
##       Balanced Accuracy : 0.8653
##
##        'Positive' Class : 1
##
```

The artificial neural network achieves approximately 85.6% accuracy, with sensitivity of 82.6% and specificity of 88.9%. Although the ANN is capable of modeling complex nonlinear relationships, it does not outperform the simpler logistic regression, decision tree, random forest, or SVM models on this dataset. This suggests that additional model complexity does not necessarily translate into better predictive performance for this problem.

```r
#ANN
ann_confus <- confusionMatrix(
  factor(ann_binpred, levels = c(0, 1)),
  factor(spotify_test$popular_binary, levels = c(0, 1)),
  positive = "1"
)
ann_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction     0     1
##          0 10075  2081
##          1  1263  9854
##
##                Accuracy : 0.8563
##                  95% CI : (0.8517, 0.8608)
##     No Information Rate : 0.5128
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.713
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8256
##             Specificity : 0.8886
##          Pos Pred Value : 0.8864
##          Neg Pred Value : 0.8288
##              Prevalence : 0.5128
##          Detection Rate : 0.4234
##    Detection Prevalence : 0.4777
##       Balanced Accuracy : 0.8571
##
##        'Positive' Class : 1
##
```

The SVM achieves approximately 86.4% accuracy, sensitivity of 82.6%, and specificity of 90.5%. Its performance is very close to logistic regression and the decision tree, demonstrating that several substantially different modeling approaches reach similar levels of predictive accuracy. The result suggests that the underlying relationship between the available song characteristics and popularity can be captured effectively by several model types.

```r
#SVM
svm_confus <- confusionMatrix(
  factor(svm_binpred, levels = c(0, 1)),
  factor(spotify_test$popular_binary, levels = c(0, 1)),
  positive = "1"
)
svm_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
```

```
## Prediction     0     1
##          0 10256  2072
##          1  1082  9863
##
##                Accuracy : 0.8645
##                  95% CI : (0.86, 0.8689)
##     No Information Rate : 0.5128
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7294
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8264
##             Specificity : 0.9046
##          Pos Pred Value : 0.9011
##          Neg Pred Value : 0.8319
##              Prevalence : 0.5128
##          Detection Rate : 0.4238
##    Detection Prevalence : 0.4703
##       Balanced Accuracy : 0.8655
##
##        'Positive' Class : 1
##
```

Overall, the six first-level models produce relatively similar results. Logistic regression has the highest overall accuracy at approximately 86.50%, followed extremely closely by the decision tree, SVM, and random forest. The decision tree produces the highest sensitivity among the first-level models at approximately 82.69%, while kNN has the lowest overall accuracy at approximately 84.91%. Because the differences between the leading models are relatively small, there is no single model that overwhelmingly dominates the others. The results instead suggest that several modeling approaches are capable of identifying meaningful patterns between song characteristics and popularity

```r
#comparing first-level models
model_comparison <- data.frame(
  Model = c("Logistic", "kNN", "Decision Tree", "Random Forest", "ANN", "SVM"),
  Accuracy = c(
    logistic_confus$overall["Accuracy"],
    knn_confus$overall["Accuracy"],
    dt_confus$overall["Accuracy"],
    rf_confus$overall["Accuracy"],
    ann_confus$overall["Accuracy"],
    svm_confus$overall["Accuracy"]
  ),
  Kappa = c(
    logistic_confus$overall["Kappa"],
    knn_confus$overall["Kappa"],
    dt_confus$overall["Kappa"],
    rf_confus$overall["Kappa"],
    ann_confus$overall["Kappa"],
    svm_confus$overall["Kappa"]
  ),
  Sensitivity = c(
    logistic_confus$byClass["Sensitivity"],
```

```
    knn_confus$byClass["Sensitivity"],
    dt_confus$byClass["Sensitivity"],
    rf_confus$byClass["Sensitivity"],
    ann_confus$byClass["Sensitivity"],
    svm_confus$byClass["Sensitivity"]
  )
)

model_comparison
```

```
##           Model  Accuracy     Kappa Sensitivity
## 1      Logistic 0.8649508 0.7303335   0.8253875
## 2           kNN 0.8491385 0.6985883   0.8207792
## 3 Decision Tree 0.8648649 0.7301393   0.8268957
## 4 Random Forest 0.8641344 0.7287481   0.8214495
## 5           ANN 0.8563142 0.7129571   0.8256389
## 6           SVM 0.8644782 0.7293688   0.8263930
```

To determine whether combining information from multiple models can improve predictive performance, we next construct a stacked model. The predicted probabilities from each of the six first-level models are used as predictors in a second-level decision tree. This allows the stacked model to learn how predictions from different algorithms can be combined rather than relying directly on the original song characteristics.

The prediction dataset is randomly divided into a 70% training set and a 30% test set for the second-level model. The training portion is used to estimate the stacked model, while the remaining observations provide an independent sample for evaluating its performance

In addition to a standard stacked model, we estimate a cost-sensitive version that places a greater penalty on failing to identify a truly popular song. This reflects the managerial objective established earlier in the analysis, where false negatives are considered more costly than false positives. The cost matrix therefore encourages the model to prioritize sensitivity, even if doing so reduces overall accuracy or specificity.

```
#creating dataframe of predictions across all models
stacked_df <- data.frame(
  logit_pred = as.numeric(logistic_predictions),
  knn_pred   = as.numeric(knn_predictions),
  ann_pred   = as.numeric(ann_predictions),
  svm_pred   = as.numeric(svm_predictions),
  tree_pred  = as.numeric(dt_predictions),
  rf_pred    = as.numeric(rf_predictions),
  y          = factor(spotify_test$popular_binary, levels = c(0, 1))
)

#splitting 70/30
ratio <- 0.7
set.seed(12345)
train_rows_stack <- sample(1:nrow(stacked_df), ratio*nrow(stacked_df))

#training and test sets for non-scaled data
stack_train <- stacked_df[train_rows_stack, ]
stack_test <- stacked_df[-train_rows_stack, ]

#cost matrix
cost_matrix <- matrix(
```

```
  c(0, 5,
    1, 0),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(predicted = c("0", "1"),
                  actual = c("0", "1")))
```

The standard stacked model achieves approximately 85.82% accuracy, with sensitivity of 82.43% and specificity of 89.40%. These results are comparable to the first-level models but do not represent an improvement over the best individual models. This indicates that combining the six model predictions through the stacked decision tree does not substantially improve general predictive accuracy in this case.

```
#Probability model (no cost matrix)
stack_model_prob <- C5.0(
  x = stack_train[, names(stack_train) != "y"],
  y = stack_train$y)

#predictions
stack_prob <- predict(
  stack_model_prob,
  stack_test[, names(stack_test) != "y"],
  type = "prob")[, "1"]

#creating binary from prediction
stack_pred_prob <- ifelse(stack_prob >= 0.5, 1, 0)

#confusion matrix
stack_confus_prob <- confusionMatrix(
  factor(stack_pred_prob, levels = c(0, 1)),
  factor(stack_test$y, levels = c(0, 1)),
  positive = "1")

stack_confus_prob
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 3037  630
##          1  360 2955
##
##                Accuracy : 0.8582
##                  95% CI : (0.8498, 0.8663)
##     No Information Rate : 0.5135
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7168
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8243
##             Specificity : 0.8940
##          Pos Pred Value : 0.8914
```

```
##          Neg Pred Value : 0.8282
##              Prevalence : 0.5135
##          Detection Rate : 0.4232
##    Detection Prevalence : 0.4748
##       Balanced Accuracy : 0.8591
##
##        'Positive' Class : 1
##
```

The cost-sensitive stacked model produces a substantially different performance profile. Its overall accuracy falls to approximately 77.63%, and specificity decreases to 58.64%, but sensitivity increases dramatically to approximately 95.62%. In practical terms, the model identifies more than 95% of the songs that are actually popular, but it does so by incorrectly classifying a larger number of not-popular songs as popular. This represents the intended tradeoff of the cost-sensitive approach: sacrificing overall accuracy and specificity in exchange for reducing false negatives.

```r
#cost sensitive classification model
stack_model_cost <- C5.0(
  x = stack_train[, names(stack_train) != "y"],
  y = stack_train$y,
  costs = cost_matrix)

#predictions
stack_pred <- predict(
  stack_model_cost,
  stack_test[, names(stack_test) != "y"])

stack_confus <- confusionMatrix(
  factor(stack_pred, levels = c(0, 1)),
  factor(stack_test$y, levels = c(0, 1)),
  positive = "1")

stack_confus
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1992  157
##          1 1405 3428
##
##                Accuracy : 0.7763
##                  95% CI : (0.7663, 0.786)
##     No Information Rate : 0.5135
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5479
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9562
##             Specificity : 0.5864
##          Pos Pred Value : 0.7093
```

```
##          Neg Pred Value : 0.9269
##              Prevalence : 0.5135
##          Detection Rate : 0.4910
##    Detection Prevalence : 0.6922
##       Balanced Accuracy : 0.7713
##
##        'Positive' Class : 1
##
```

The final comparison demonstrates an important tradeoff between overall predictive accuracy and the managerial objective of identifying popular songs. Among the standard models, logistic regression, the decision tree, random forest, and SVM all achieve approximately 86% accuracy, while their sensitivities remain near 82%. The standard stacked model does not outperform these individual models. In contrast, the cost-sensitive stacked model lowers overall accuracy to approximately 77.6% but increases sensitivity to approximately 95.6%. Therefore, if overall classification accuracy is the primary objective, logistic regression or another leading first-level model would be preferable. However, if the business places substantially greater value on identifying as many potentially popular songs as possible, the cost-sensitive stacked model may be more appropriate despite its substantially higher false-positive rate

```r
#adding stacked model for model comparison
model_comparison <- rbind(
  model_comparison,
  data.frame(
    Model = "Stacked (Prob, 0.5 threshold)",
    Accuracy = stack_confus_prob$overall["Accuracy"],
    Kappa = stack_confus_prob$overall["Kappa"],
    Sensitivity = stack_confus_prob$byClass["Sensitivity"]),
  data.frame(
    Model = "Stacked (Cost-sensitive)",
    Accuracy = stack_confus$overall["Accuracy"],
    Kappa = stack_confus$overall["Kappa"],
    Sensitivity = stack_confus$byClass["Sensitivity"])
)
model_comparison
```

```
##                                  Model  Accuracy     Kappa Sensitivity
## 1                             Logistic 0.8649508 0.7303335   0.8253875
## 2                                  kNN 0.8491385 0.6985883   0.8207792
## 3                        Decision Tree 0.8648649 0.7301393   0.8268957
## 4                        Random Forest 0.8641344 0.7287481   0.8214495
## 5                                  ANN 0.8563142 0.7129571   0.8256389
## 6                                  SVM 0.8644782 0.7293688   0.8263930
## Accuracy  Stacked (Prob, 0.5 threshold) 0.8582068 0.7167981   0.8242678
## Accuracy1      Stacked (Cost-sensitive) 0.7762819 0.5478839   0.9562064
```